

VisionGuard AI: A High-Performance Comparative Medical Image Segmentation and Explainability Platform

Anuj Meena

Abstract

This report presents the design, mathematical formulation, and runtime optimization of VisionGuard AI, an enterprise-grade medical image segmentation benchmarking framework. The platform evaluates three distinct computer vision paradigms: a convolutional neural network (U-Net) with skip connections, a Vision Transformer (SegFormer) with a hierarchical encoder and lightweight MLP decoder, and a zero-shot promptable foundation network (Segment Anything Model - SAM). To address clinical explainability requirements, we implement Grad-CAM (Gradient-weighted Class Activation Mapping) layers that backpropagate class logits to map spatial attention matrices. To resolve real-time CPU latency bottlenecks, the U-Net computational graph is exported and optimized under ONNX Runtime, achieving a 2.98x latency speedup (38.65 ms vs 115.42 ms native PyTorch). We present the full mathematical definitions of the models, the loss functions, the ONNX graph modifications, and the containerized microservice APIs.

I. INTRODUCTION

Medical image segmentation—isolating anatomical features such as tumor boundaries, lung lobes, or cardiac regions—is critical to computer-assisted diagnostics. While deep learning has enabled automated segmentation, production deployments face two major challenges:

- 1) **Explainability:** Medical practitioners cannot trust black-box model decisions. Saliency maps indicating where models focus are legally and operationally required.
- 2) **Inference Latency:** Deploying complex vision models in hospital settings, often on hardware without high-end GPUs, demands high CPU throughput.

VisionGuard AI solves these issues by comparative benchmarking of three distinct model philosophies (U-Net, SegFormer, SAM), integrating Grad-CAM explainability, and deploying optimized subgraphs via ONNX Runtime.

II. MATHEMATICAL FORMULATIONS & MODEL ARCHITECTURES

The platform implements three separate segmentation pipelines, as described below.

A. U-Net Convolutional Network

The U-Net architecture consists of a contracting path (encoder) and an expansive path (decoder).

- **Contracting Path:** Consists of repeated application of two 3×3 convolutions, each followed by a Rectified Linear Unit (ReLU) and a 2×2 max pooling operation with stride 2 for downsampling.
- **Expansive Path:** Consists of an upsampling of the feature map followed by a 2×2 convolution (“up-convolution”) that halves the number of feature channels, a concatenation with the correspondingly cropped feature map from the contracting path, and two 3×3 convolutions, each followed by a ReLU.

The skip connections transfer high-resolution localized features directly from the encoder to the decoder, preventing spatial degradation.

B. SegFormer Vision Transformer

SegFormer consists of a hierarchical Transformer encoder that outputs multiscale features and a lightweight All-MLP decoder. Unlike standard ViTs, it avoids positional encodings by using a Mix-FFN layer:

$$x_{out} = \text{MLP}(\text{GELU}(\text{Conv}_{3 \times 3}(\text{MLP}(x_{in})))) + x_{in} \quad (1)$$

The MLP decoder aggregates multi-level features from the four transformer blocks, normalizes channel dimensions, upsamples them to original resolutions, and concatenates them to output the mask probability map.

C. Segment Anything Model (SAM)

SAM utilizes a heavy ViT image encoder to compute high-dimensional feature embeddings, a prompt encoder mapping point clicks, and a lightweight mask decoder that combines the embeddings to output segmentation boundaries.

D. Loss Function Formulations

Models are trained using a combination of Binary Cross-Entropy (BCE) and Dice Loss. Let Y be the ground truth binary mask, and $\hat{Y}$ be the predicted probability map:

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \quad (2)$$

$$\mathcal{L}_{\text{Dice}} = 1 - \frac{2 \sum_{i=1}^N y_i \hat{y}_i + \epsilon}{\sum_{i=1}^N y_i + \sum_{i=1}^N \hat{y}_i + \epsilon} \quad (3)$$

where $\epsilon = 1e - 5$ is a smoothing coefficient. The joint loss is formulated as:

$$\mathcal{L}_{\text{Joint}} = \mathcal{L}_{\text{BCE}} + \mathcal{L}_{\text{Dice}} \quad (4)$$

E. Grad-CAM Saliency Math

To generate Grad-CAM heatmaps, we compute the gradient of the score for class c , y^c (before softmax), with respect to feature map activations A^k of the final convolutional layer:

$$\alpha_k^c = \frac{1}{Z} \sum_i \sum_j \frac{\partial y^c}{\partial A_{i,j}^k} \quad (5)$$

where Z is the height $\times$ width of the feature map. We then compute a weighted combination of forward activation maps, followed by a ReLU operation to capture features that positively correlate with the target:

$$L_{\text{Grad-CAM}}^c = \text{ReLU} \left(\sum_k \alpha_k^c A^k \right) \quad (6)$$

III. RUNTIME COMPILATIONS & OPTIMIZATION

To optimize U-Net CPU execution, we export the PyTorch model to ONNX format. The model's symbolic graph is compiled using constant folding, which pre-calculates static nodes during compilation. Furthermore, dynamic axes are specified for batch dimensions to allow varying batch sizes:

```
torch.onnx.export(
    model, dummy_input, onnx_path,
    export_params=True, opset_version=15,
    do_constant_folding=True,
    input_names=['input'], output_names=['output'],
    dynamic_axes={'input': {0: 'batch_size'}, 'output': {0: 'batch_size'}}
)
```

IV. PERFORMANCE BENCHMARKS

Benchmarks were run on a single-threaded CPU environment (Image size = 256x256, Batch Size = 1).

The optimized ONNX Runtime model demonstrates a **2.98x speedup factor** compared to the native PyTorch FP32 baseline.

TABLE I
INFERENCE LATENCY COMPARISONS

Runtime Engine	Device	Average Latency (ms)	Throughput (FPS)
PyTorch FP32 Native	CPU	115.42 ms	8.66
ONNX Runtime (Optimized)	CPU	38.65 ms	25.87

V. PRODUCTION ENGINEERING & DEVOPS

The system is constructed with a clean, unified structure:

- 1) **Microservices:** FastAPI exposes segmentation, explainability, and SAM coordinates endpoints.
- 2) **Dashboards:** Streamlit handles image uploading, SAM click coordinate captures, and Plotly latency overlays.
- 3) **Verification Framework:** Supports ‘make verify’ running linting, type checks, unit tests, and JSON benchmark logging to ‘reports/latest.json’.
- 4) **Dockerization:** Containerized using a multi-stage, lightweight Dockerfile.